WEEK 4 · LESSON 24 OF 24 · HANDOUT

Final Project: Predict Trip Fares and Certificate

Study time: **about 90 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=nKW8Ndu7Mjw — open the lesson online to watch inside the course.

Your goal in this lesson: Combine Python, data handling, statistics, linear algebra and calculus to build, train and evaluate a multiple linear regression model — and earn your certificate.

The goal

Improve last lesson's fare model by using **more than one feature**: distance **and** rain (and, if you like, the hour). You will write the whole workflow yourself in one Colab notebook, in the order a professional would. Re-watch the workflow video if you need a reminder of the stages.

Part 1 — Data (Weeks 1–2)

Generate the moto-trip data with the Week 2 project cell (including the data problems), then clean it exactly as you did in that project and write a short cleaning log.

Part 2 — Explore (Weeks 2–3)

- Describe `fare_rwf`: mean, median, SD.
- Correlation matrix of the number columns (convert `raining` with `.astype(int)`).
- A scatter plot of distance vs fare, coloured by rain.

Part 3 — Prepare the matrix (Week 3)

Build a feature matrix X with one row per trip and these columns: `distance_km`, `raining` (0/1). Standardise each column (Lesson 17), and keep the means and SDs so you can apply the same scaling to new trips.

```
features = ["distance_km", "raining"]
X = clean[features].astype(float).to_numpy()
y = clean["fare_rwf"].to_numpy()

rng = np.random.default_rng(1)
idx = rng.permutation(len(X))
cut = int(0.8 * len(X))
train_idx, test_idx = idx[:cut], idx[cut:]

mean, std = X[train_idx].mean(axis=0), X[train_idx].std(axis=0)
X_scaled = (X - mean) / std
X_train, X_test = X_scaled[train_idx], X_scaled[test_idx]
y_train, y_test = y[train_idx], y[test_idx]
```

Use only the **training** rows to compute the mean and SD — otherwise information from the test set leaks into training.

Part 4 — Train with gradient descent (Week 4)

The model is now a matrix multiplication: **prediction = X @ w + b**.

```
w = np.zeros(X_train.shape[1])
b = 0.0
lr = 0.1
for epoch in range(2000):
    error = X_train @ w + b - y_train
    grad_w = 2 * X_train.T @ error / len(y_train)
    grad_b = 2 * error.mean()
    w -= lr * grad_w
    b -= lr * grad_b
print(w.round(1), round(b, 1))
```

Notice `X_train.T @ error`: the transpose (Lesson 14) computes the gradient for every weight in one line.

Check your answer with NumPy's least-squares solver:

```
A = np.column_stack([X_train, np.ones(len(X_train))])
print(np.linalg.lstsq(A, y_train, rcond=None)[0].round(1))
```

Part 5 — Evaluate

1. Test MAE and R^2 for the new model.
2. Compare with the distance-only model from Lesson 23. Did adding rain help? By how much?
3. Convert the weights back to real units: divide each weight by its feature's SD. How many RWF per km? How many extra RWF when it rains?
4. Predict the fare for a 6 km trip in the rain. (Scale the new trip with the **training** mean and SD.)

Part 6 — Report (one page)

- What the model predicts and how well (MAE in RWF, R^2).
- What each feature adds to the fare, in plain words.
- One chart (predicted vs real fares on the test set, with a diagonal line).
- Two limitations — for example, the data is simulated, and real fares also depend on traffic and negotiation.

Marking guide (self-check)

Area	Excellent work
Data	Problems found and fixed, with a cleaning log
Exploration	Correct statistics, clear labelled charts
Model	Gradient descent matches NumPy's answer
Evaluation	Honest test-set results compared with a baseline
Communication	A manager understands the report in five minutes

Getting your certificate

When you have completed **every lesson** in the course, your **WAMUGII Certificate of Completion** unlocks on the course page, and you receive a congratulations email. Add it — and a link to your project notebook — to your CV and

LinkedIn.

What you can do now

You can write Python, clean and explore data, use the maths behind AI — vectors, matrices, statistics, probability and derivatives — and you have trained a machine-learning model from scratch. Next: **Machine Learning Fundamentals**, where you will use scikit-learn to build many kinds of models quickly and evaluate them like a professional.

Remember this: A real model is the sum of everything in this course: clean data, good statistics, a matrix of features, a loss, gradients and honest evaluation.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6 in one notebook. Then add the `hour` feature to the model. Does it help? Explain your result using the correlation matrix from Part 2.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. In multiple linear regression with a feature matrix X, how are predictions computed?
   A. X / w
   B. X + w
   C. w only
   D. X @ w + b
2. Why compute the scaling mean and SD from the training rows only?
   A. It is faster
   B. To stop information from the test set leaking into training
   C. It does not matter
   D. The test set has no numbers
3. What does `X_train.T @ error` compute in the training loop?
   A. The predictions
   B. The test score
   C. The gradient part for every weight at once
   D. The mean of X
4. How do you check that your gradient descent is correct?
   A. Compare its weights with NumPy's least-squares solution
   B. You cannot check
   C. Run it once
   D. Look at the colours
5. Adding the rain feature lowers the test MAE (for example, from 160 to 120 RWF). What does that show?
   A. Rain information helps the model predict fares better
   B. The model is broken
   C. MAE should go up
   D. Rain is not related to fares
6. Which is an honest limitation to include in the report?
   A. Regression always works
   B. The data is simulated; real fares also depend on traffic and negotiation
   C. There are no limitations
   D. The model is perfect

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.